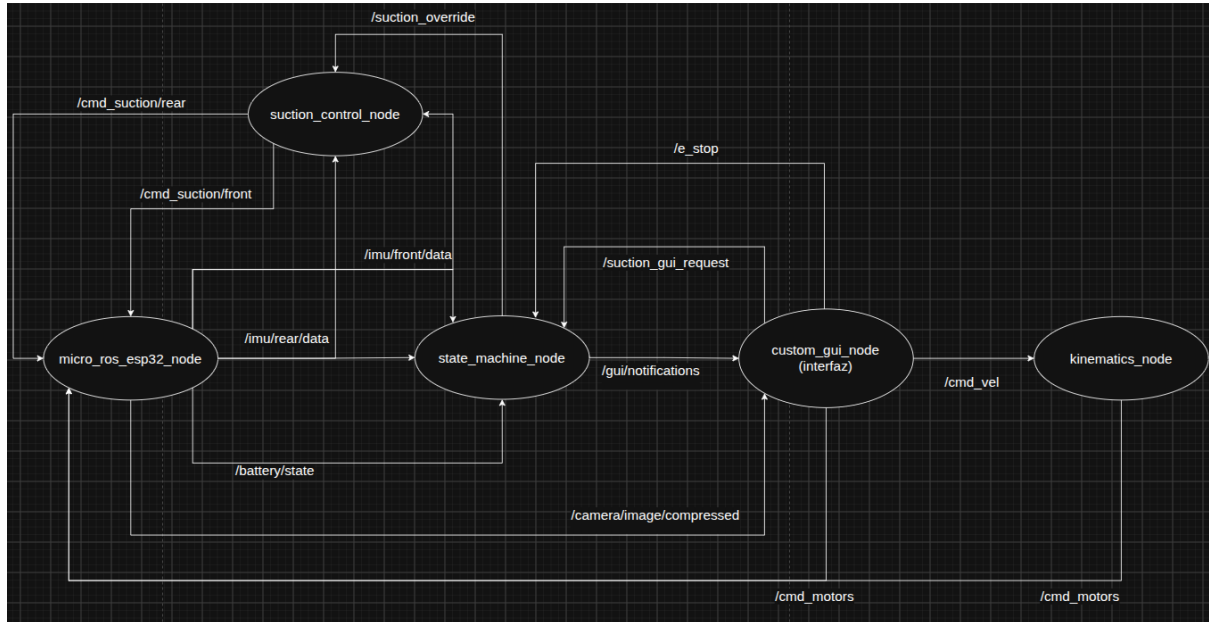


Robot Carro Trepador — Arquitectura ROS 2 Humble

Resumen de paquetes, nodos y tópicos del sistema de control. El operador interactúa con la GUI (custom_gui_node); los comandos viajan por tópicos ROS 2 hasta el firmware en la ESP32-S3, que acciona los motores y turbinas.

1. Flujo de comunicación del robot



2. Nodos del sistema

Nodo	Paquete	Tipo	Descripción / Importancia
micro_ros_esp32_node	firmware/ (micro-ROS)	Bajo nivel (Embebido)	Nodo embebido en ESP32-S3 vía Wi-Fi/micro-ROS. Convierte comandos de software en señales PWM para motores N20 y turbinas BLDC; publica lecturas de las dos IMUs, batería y cámara.
state_machine_node	carro_trepador_control	Alto nivel (Supervisor)	Supervisor de seguridad central. Valida órdenes del operador (rechaza apagar succión si el robot está en pared), vigila batería y publica notificaciones a la GUI.
suction_control_node	carro_trepador_control	Alto nivel (Control)	Traduce la inclinación en grados de cada IMU a un porcentaje de potencia para cada turbina. Evita sobrecalentamiento y ahorra batería en superficies planas.
kinematics_node	carro_trepador_control	Alto nivel (Matemática)	Convierte /cmd_vel (velocidad lineal/angular) en velocidades individuales para las ruedas izquierda y derecha mediante cinemática diferencial.
custom_gui_node	carro_trepador_teleop	Alto nivel (Interfaz)	Interfaz gráfica PyQt en la PC. Permite controlar el robot con deslizadores y botones, ver la cámara en vivo, ejecutar pruebas de motores, publicar /e_stop y recibir alertas.

3. Tópicos de comunicación

Nota sobre /e_stop: tópico independiente tipo std_msgs/Bool. Cualquier nodo puede publicarlo. state_machine_node decide el comportamiento según el estado actual del robot (en pared → succión)

máxima + tracción 0; en suelo → todo apagado). El firmware implementa un watchdog local: si no recibe heartbeat por más de 500 ms, ejecuta la parada de emergencia sin esperar a ROS 2.

Tópico	Tipo de mensaje	Publicador	Suscriptor	Función
/cmd_vel	geometry_msgs/msg/Twist	custom_gui_node	kinematics_node	Velocidad lineal y angular deseada del robot.
/cmd_motors	std_msgs/msg/Float32MultiArray	kinematics_node custom_gui_node (diagnóstico)	micro_ros_esp32_node	Velocidades individuales de cada rueda N20. En modo diagnóstico, la GUI puede publicar directamente; el state_machine_node bloquea este modo durante operación normal.
/cmd_suction/front /cmd_suction/rear	std_msgs/msg/Float32	suction_control_node	micro_ros_esp32_node	Potencia de turbina (0–100%) para módulo frontal y trasero de forma independiente.
/imu/front/data /imu/rear/data	sensor_msgs/msg/Imu	micro_ros_esp32_node	state_machine_node suction_control_node	Orientación y aceleración de cada módulo. Permite detectar transiciones piso-pared y calcular el vector de gravedad por segmento.
/suction_gui_request	std_msgs/msg/Float32	custom_gui_node	state_machine_node	Petición manual del operador para modificar el nivel de succión. No se aplica directamente; pasa por el supervisor de seguridad.
/suction_override	std_msgs/msg/Float32	state_machine_node	suction_control_node	Orden prioritaria de succión emitida por la máquina de estados. Fuerza la potencia de turbinas ante riesgo de caída.
/e_stop	std_msgs/msg/Bool	custom_gui_node	state_machine_node	Parada de emergencia. True = detener tracción y forzar succión máxima si el robot está en pared; apagado total si está en suelo.
/battery/state	sensor_msgs/msg/BatteryState	micro_ros_esp32_node	state_machine_node	Voltaje de la batería LiPo 4S. Activa alertas de retorno o emergencia antes de un apagado repentino en altura.
/camera/image/compressed	sensor_msgs/msg/CompressedImage	micro_ros_esp32_node	custom_gui_node	Stream de video JPG comprimido desde la OV2640. Formato comprimido para no saturar el ancho de banda Wi-Fi.
/gui/notifications	std_msgs/msg/String	state_machine_node	custom_gui_node	Alertas de texto: bloqueos de seguridad, modo activo, batería baja, etc.

4. Quién publica, en qué tópico y quién escucha

Nota: /cmd_motors tiene dos publicadores posibles (kinematics_node en operación normal; custom_gui_node en modo diagnóstico). El state_machine_node bloquea el modo diagnóstico cuando el robot está en operación activa para evitar conflictos.

Publicador	Tópico	Suscriptor
custom_gui_node	/cmd_vel	kinematics_node
custom_gui_node	/cmd_motors	micro_ros_esp32_node (diagnóstico)
custom_gui_node	/suction_gui_request	state_machine_node
custom_gui_node	/e_stop	state_machine_node
kinematics_node	/cmd_motors	micro_ros_esp32_node
state_machine_node	/suction_override	suction_control_node
state_machine_node	/gui/notifications	custom_gui_node
suction_control_node	/cmd_suction/front	micro_ros_esp32_node
suction_control_node	/cmd_suction/rear	micro_ros_esp32_node
micro_ros_esp32_node	/imu/front/data	state_machine_node, suction_control_node
micro_ros_esp32_node	/imu/rear/data	state_machine_node, suction_control_node
micro_ros_esp32_node	/battery/state	state_machine_node
micro_ros_esp32_node	/camera/image/compressed	custom_gui_node

5. Paquetes del proyecto

Paquete	Descripción
carro_trepador_interfaces	Define mensajes personalizados (.msg). Actúa como "contrato" de comunicación. Actualmente todos los tópicos usan tipos estándar de ROS 2; este paquete queda disponible para mensajes personalizados futuros. No ejecuta ningún nodo.
carro_trepador_description	URDF/Xacro con la geometría física del robot articulado de dos módulos (links, joints, mallas STL). Sin algoritmos. Base para simulación en Gazebo y visualización en RViz2.
carro_trepador_control	Contiene kinematics_node, suction_control_node y state_machine_node. Calcula cinemática diferencial, ajusta potencia de succión según las IMUs y actúa como supervisor anti-caídas.
carro_trepador_teleop	Contiene custom_gui_node: interfaz gráfica PyQt para teleoperación, cámara en vivo, diagnóstico de motores y alertas. Publica /e_stop mediante botón de emergencia.
carro_trepador_bringup	Archivos launch que arrancan toda la arquitectura con un solo comando. Pendiente de crear con: <code>ros2 pkg create carro_trepador_bringup --build-type ament_cmake</code>
firmware/ (micro-ROS)	Código C++ para ESP32-S3 en PlatformIO. Implementa micro_ros_esp32_node: lee IMUs (MPU6050 ×2), batería, cámara OV2640 y aplica PWM a motores N20 y ESC de turbinas BLDC vía Wi-Fi.

6. Comandos de prueba

Compilar y cargar el workspace:

```
cd ~/Carro_trepador/ros2_ws && source /opt/ros/humble/setup.bash && colcon build --symlink-install && source install/setup.bash
```

Lanzar todo el sistema:

```
ros2 launch carro_trepador_bringup bringup.launch.py
```

Visualización en RViz2:

```
ros2 run rviz2 rviz2 # Fixed Frame: base_link → Add RobotModel → Description
Topic: /robot_description
```